

Setup & Installation Guide

This document provides step-by-step instructions for installing, configuring, running, testing, and troubleshooting the **Hostel Outing Permission & Approval Management System**.

1. Prerequisites

Before installing the application, ensure the following software components are installed:

- **Node.js:** v18.0.0 or higher (nodejs.org (<https://nodejs.org/>))
- **npm:** v9.0.0 or higher (bundled with Node.js)
- **Python:** v3.10 or higher ([python.org](https://www.python.org/) (<https://www.python.org/>))
- **Git:** (git-scm.com (<https://git-scm.com/>))

Verify versions in your terminal:

```
node -v  
npm -v  
python --version
```

2. Project Structure

```

c:\Data\Inter Assign\
├─ hostel_outing.db           # Primary SQLite Persistent Database
├─ README.md                 # Project overview & quickstart
├─ pytest.ini                # Pytest configuration
├─ docker-compose.yml        # Docker deployment setup
├─ backend/                  # Python FastAPI Backend
│   ├─ app/                  # API, Models, Schemas, Services
│   ├─ tests/                # Pytest suite (83 test cases)
│   ├─ seed_data.py          # Database seeder script
│   ├─ requirements.txt      # Python dependencies
│   └─ venv/                 # Python virtual environment
├─ frontend/                 # React TypeScript Frontend
│   ├─ src/                  # Components, Pages, Router, Types
│   ├─ package.json          # Dependencies & scripts
│   └─ vite.config.ts        # Vite config & dev server proxy
├─ e2e/                      # Playwright End-to-End Test Suite
│   ├─ playwright.config.ts  # Playwright configuration
│   └─ *.spec.ts             # E2E test scripts
├─ docs/                     # Documentation suite
│   ├─ architecture.md
│   ├─ design.md
│   ├─ user-guide.md
│   ├─ api.md
│   └─ setup.md (This file)
└─ evidence/                 # Test & quality evidence reports
    ├─ ai-change-loop.md
    └─ red-run.md

```

3. Backend Setup

Open a terminal in the root workspace directory (c:\Data\Inter Assign).

Create Virtual Environment

```
python -m venv backend/venv
```

4. Python Environment Activation

Windows (PowerShell)

```
.\backend\venv\Scripts\Activate.ps1
```

Linux / macOS

```
source backend/venv/bin/activate
```

5. Dependency Installation

Install backend Python packages listed in `backend/requirements.txt`:

```
.\backend\venv\Scripts\python.exe -m pip install -r backend/requirements.txt
```

Backend Dependency Summary

- **fastapi**: Async web framework (`>=0.110.0`)
- **uvicorn[standard]**: ASGI production server (`>=0.28.0`)
- **sqlalchemy**: Relational ORM (`>=2.0.28`)
- **pydantic & pydantic-settings**: Data validation (`>=2.6.4`)
- **python-jose & passlib[bcrypt]**: JWT authentication & password hashing
- **pytest & pytest-asyncio & httpx**: Testing suite

6. Database Setup

- **Default Engine**: SQLite for development and testing.
- **Database File Location**: `c:\Data\Inter Assign\hostel_outing.db`.
- **Automatic Initialization**: Tables are automatically created on backend startup via `Base.metadata.create_all(bind=engine)`.
- **PostgreSQL Compatibility**: Set `DATABASE_URL` environment variable for production deployments (e.g. `postgresql://user:password@localhost:5432/hostel_db`).

7. Seed Data Execution

The project includes an **idempotent, non-destructive** database seeder: `backend/seed_data.py`.

Run the seeder from the workspace root:

```
.\backend\venv\Scripts\python.exe backend/seed_data.py
```

Seeded Demo Accounts (All passwords: Hostel@123)

Role	Name	Email	Password	Scope
Student	Arjun Raj	student.a@hostelapp.local	Hostel@123	CSE / A Block
Student	Nithya S	student.b@hostelapp.local	Hostel@123	ECE / B Block
Student	Rahul Menon	student.c@hostelapp.local	Hostel@123	CSE / C Block
HOD	Dr. Arun Kumar	hod.cse@hostelapp.local	Hostel@123	CSE Department
HOD	Dr. Priya Sharma	hod.ece@hostelapp.local	Hostel@123	ECE Department
Warden	Mr. Rajesh Kumar	warden.a@hostelapp.local	Hostel@123	A Block
Warden	Ms. Meena Krishnan	warden.b@hostelapp.local	Hostel@123	B Block
Warden	Mr. Suresh Kumar	warden.c@hostelapp.local	Hostel@123	C Block
Watchman	Mr. Suresh B	watchman@hostelapp.local	Hostel@123	Main Gate Desk

8. Running FastAPI Backend Server

Execute Uvicorn from the root directory (c:\Data\Inter Assign):

```
.\backend\venv\Scripts\python.exe -m uvicorn backend.app.main:app --reload --port 8000
```

- **Backend Base URL:** <http://127.0.0.1:8000>
- **Interactive Swagger Docs:** <http://127.0.0.1:8000/docs>
- **ReDoc Documentation:** <http://127.0.0.1:8000/redoc>

9. Frontend Setup

Navigate to the `frontend/` directory and install Node.js dependencies:

```
cd frontend
npm install
```

10. Running Vite Frontend Dev Server

From the `frontend/` directory, start the Vite development server:

```
npm run dev
```

- **Frontend URL:** `http://localhost:5173`
- **Vite Proxy:** Automatically proxies `/api` requests to `http://127.0.0.1:8000/api`.

11. Running Backend Pytest Suite

Run the full automated backend test suite from the root directory:

```
.\backend\venv\Scripts\pytest.exe -v
```

*[!NOTE] Pytest executes against an isolated in-memory SQLite database (`sqlite:///memory:`). Running backend tests will **never modify or erase** your development database `hostel_outing.db`. All **83 test cases** pass cleanly.*

12. Running Playwright End-to-End Tests

1. Ensure both Backend (`http://127.0.0.1:8000`) and Frontend (`http://localhost:5173`) servers are running.
2. Open a separate terminal, navigate to `e2e/`, and execute:

```
cd e2e
npx playwright test
```

- **View Test Report:** `npx playwright show-report`

13. Building Frontend for Production

Verify TypeScript types and compile production static assets:

```
cd frontend
npm run build
```

- **Output Assets Directory:** `frontend/dist/`
 - **Local Production Preview:** `npm run preview`
-

14. Environment Variables

Variable Name	Default Value	Description
DATABASE_URL	sqlite:///c:/Data/Inter Assign/hostel_outing.db	SQLAlchemy database connection string
SECRET_KEY	secret-key-for-jwt-development-only-change-in-prod	Secret key for signing JWT tokens
ALGORITHM	HS256	JWT signing algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	480	JWT token validity duration (minutes)

15. Troubleshooting Common Issues

Issue 1: `http proxy error: connect ECONNREFUSED 127.0.0.1:8000`

- **Cause:** Frontend Vite server is running, but backend FastAPI server is stopped.
- **Fix:** Start the backend server using `.\backend\venv\Scripts\python.exe -m uvicorn backend.app.main:app --reload --port 8000`.

Issue 2: `sqlite3.OperationalError: no such column: gate_logs.delay_minutes`

- **Cause:** An outdated `hostel_outing.db` file exists from an earlier schema version.
- **Fix:** Remove the stale database file (Remove-Item `hostel_outing.db` -Force) and re-run `.\backend\venv\Scripts\python.exe backend/seed_data.py`.

Issue 3: `ModuleNotFoundError: No module named 'backend'`

- **Cause:** Running python commands from inside `backend/` without specifying module path.
 - **Fix:** Always execute commands from the root directory (`c:\Data\Inter Assign`).
-

16. Demo Workflow (5-Minute Demonstration Guide)

Follow this 5-minute walkthrough to demonstrate the complete system lifecycle:

1. **Student Submission:** Log in as `student.a@hostelapp.local` (Hostel@123). Submit an outing request for today (09:00 to 17:00). Status is `PENDING_HOD`.
2. **HOD Academic Review:** Log in as `hod.cse@hostelapp.local`. Navigate to dashboard, view request, click **Approve**. Status advances to `PENDING_WARDEN`.
3. **Warden Parent Verification:** Log in as `warden.a@hostelapp.local`. Click **Confirm Parent Approval** (simulating phone call), then click **Approve Outing**. Status becomes `APPROVED`.
4. **Gate Exit:** Log in as `watchman@hostelapp.local`. Search student `CSE2027001` or Outing ID `#OUT-4`. Click **Record Exit**. Status becomes `EXITED`.
5. **Gate Return & Late Detection:** Click **Record Return**. System evaluates return time against expected return, sets status to `COMPLETED` or `LATE_RETURN`, logs delay minutes, and appends audit history.
6. **Audit History Inspection:** Log in as Student, HOD, or Warden, open **History**, and click **View Audit Timeline** to demonstrate the immutable audit log.